

6. Heap sort

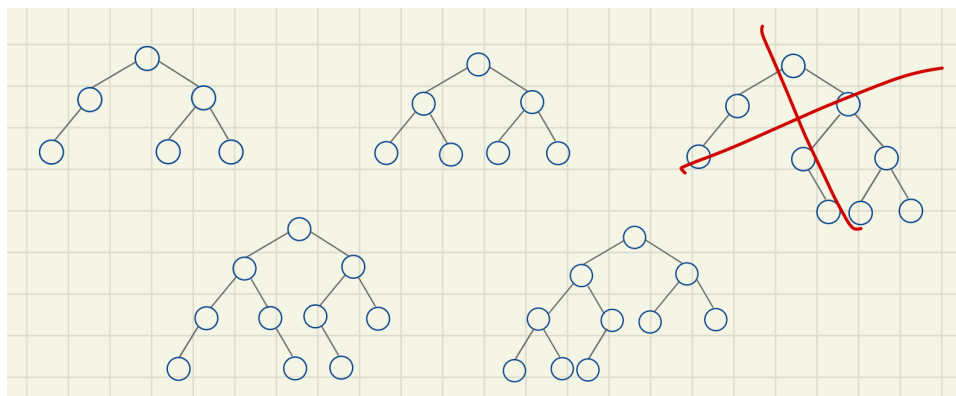


Lezione 15 - @October 27, 2025

Heap sort

Albero binario quasi completo

Un albero **binario quasi completo** se è completo fino al penultimo livello



Proprietà: ogni nodo di altezza $< h - 1$ possiede entrambi i figli, con h altezza dell'albero.

fissato h

- albero completo fino a profondità $h - 1$
- almeno un nuovo nodo di profondità h

$$\text{numero_min_nodi} = \begin{cases} \text{albero completo fino a profondità } h - 1 & 2^h - 1 \\ \text{almeno un nodo di prof. } h & 1 \end{cases}$$

$$2^h - 1 + 1 = 2^h$$

numero_max_nodi = albero completo con profondità h

$$2^{h+1} - 1$$

Si può impostare l'equazione:

$$\begin{aligned} 2^h &\leq n \leq 2^{h+1} - 1 \\ 2^h &\leq n < 2^{h+1} \end{aligned}$$

$$h \leq \log_2 n$$

$$\log_2 n < h + 1$$

Dunque:

$$h = \lfloor \log_2 n \rfloor$$

Heap

Il **max-heap** è un albero binario quasi completo in cui la chiave contenuta in ciascun nodo è maggiore o uguale delle chiavi contenute nei figli

- Per comodità consideriamo heap in cui le foglie dell'ultimo livello si trovano più a sinistra possibile
- Nei disegni ci sono chiavi numeriche. Ci sono anche altri campi!
 - Per comodità si considerano **heap** in cui le foglie dell'ultimo livello si trovano più a sinistra possibile.

»

esempi slides

Procedura **risistema** (**fixHeap**) uno heap con radice “sbagliata”

```
PROCEDURA risistema(Heap H)
  v ← H                // v: nodo in esame
  x ← v.chiave          // x: chiave da sistemare
  y ← v.altricampi      // y: campi associati a x
  daCollocare ← true
  DO
    IF v è una foglia THEN
      daCollocare ← false
    ELSE
      u ← figlio di v di valore max
      IF u.chiave > x THEN
        v.chiave ← u.chiave
        v.altricampi ← u.altricampi
        v ← u
      ELSE
        daCollocare ← false
  WHILE daCollocare
  v.chiave ← x
  v.altricampi ← y
```

Questo algoritmo, per un heap di altezza h , esegue $\Theta(h)$ confronti.

Costruzione Heap

dato un albero binario quasi completo:

- strategia top-down - tecnica *divide-et-impera*
- bottom-up → da sottoalberi piccoli a quelli più grandi

Soluzione 1: *divide-et-impera*

- albero vuoto → è già uno heap
- albero non vuoto → algoritmo:

```
PROCEDURA creaHeap(Albero T)
  IF T != albero vuoto THEN
    creaHeap(T.sx)
    creaHeap(T.dx)
    risistema(T)
```

Problemi con questa implementazione:

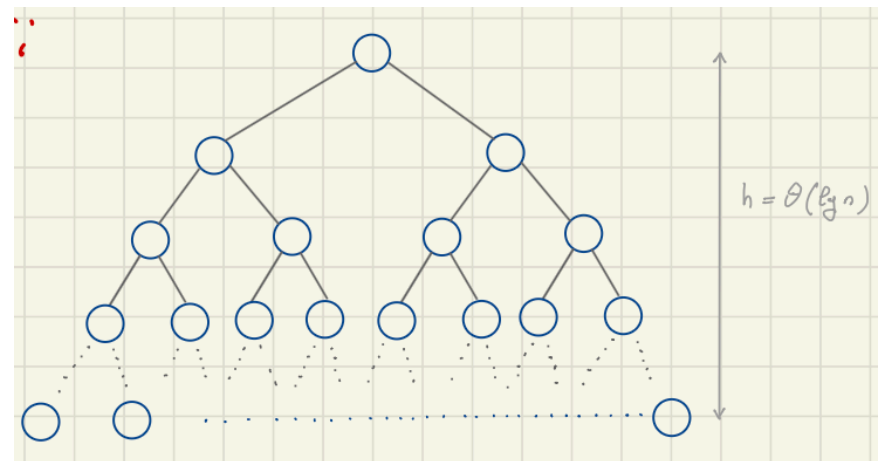
- uso dello stack

Soluzione 2: partire dalle foglie

```
PROCEDURA creaHeap(Albero T)
  h <- altezza di T
  FOR p <- h DOWN TO 0 DO
    FOREACH nodo x di profondità p DO
      risistema(sottoalbero di radice x)
```

Analisi delle prestazioni immediata

- `risistema` esegue $O(h')$ confronti dove h' è l'altezza dell'albero da risistemare
- $h' \leq h \rightarrow$ ogni chiamata fa al più $O(h)$ confronti
- `num_cfr` è al più:



$$n \cdot O(h) \leq O(n \log n)$$
$$h = \lfloor \log_2 n \rfloor$$

Analisi migliorata

- $\sum_{i=0}^n 2^i = 2^{n+1} - 1$
- $\sum_{i=0}^n i \cdot 2^i = (n - 1)2^{n+1} + 2$

verifica per induzione:

- per $i = 0$:

$$0 - 1 \cdot 2 + 2 = 0$$

- $i > 0$:

$$\begin{aligned} \sum_{i=0}^n i \cdot 2^i &= \sum_{i=0}^n -1i \cdot 2^i + n2^n \text{ (hp)} \Rightarrow (n - 2)2^n + 2 + n2^n \\ &= n2^n - 2^{n+1} + 2 + n2^n = (n - 1)2^{n+1} + 2 \end{aligned}$$

partendo dal basso, considerando un albero di profondità p , costituito da sotto-alberi di altezza $h - p$, il numero di confronti per ogni albero è di $\Theta(h - p)$.

calcolo confronti totali

$$\sum_{p=0}^h 2^p \cdot \Theta(h - p)$$

sviluppando l'espressione:

$$\begin{aligned} \sum_{p=0}^h 2^p \cdot (h - p) &= h \sum_{p=0}^h 2^p - \sum_{p=0}^h p2^p \\ &= h2^{h+1} - h - (h - 1)2^{h+1} - 2 = h2^{h+1} - h - h2^{h+1} - 2 \\ &= 2^{h+1} - h - 2 \end{aligned}$$

se $h \approx \log_2 n \rightarrow C(n) = \Theta(n)$

Algoritmo finale

ALGORITMO *heapSort*(Array A) $\rightarrow$ lista
 crea uno Heap H a partire da A (crea albero quasi completo...come?)
 $x \leftarrow$ lista vuota
WHILE $H \neq \emptyset$ **DO**
 - rimuovi da H il valore alla radice e aggiungilo all'inizio di x
 - rimuovi la foglia più a destra dell'ultimo livello e collocane
 il valore alla radice
 - *risistema*(H)
RETURN x

- crea Heap $\Theta(n)$ confronti
- caso peggiore per *risistema* $O(\log_n)$
- quante iterazioni: n
- nel ciclo, confronti totali: $O(n \log n)$
- confronti totali: $\Theta(n) + O(n \log n) = O(n \log n)$

Ma quanto costa creare un albero binario *quasi completo*?



Lezione 16 - @October 29, 2025

Dalla lezione 15 - **heap** *letteralmente mucchio*

non conviene usare l'approccio *top-down* poiché richiede memoria aggiuntiva (stack)

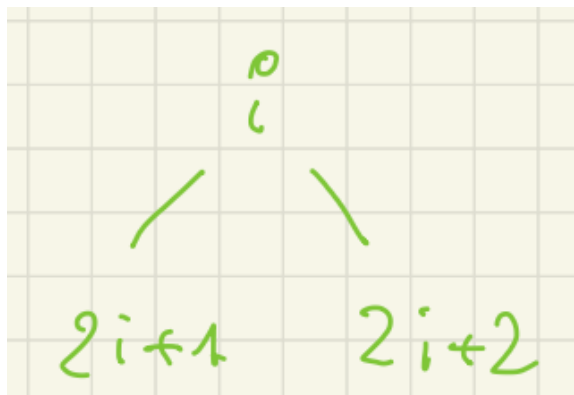
procedura **risistema** con uno heap a radice *sbagliata*

Costruzione di un Heap *bottom up*

```
PROCEDURA creaHeap(Albero T)
  h <- altezza di T
  FOR p <- h DOWNTO 0 DO
    FOREACH nodo x di profondità p DO
      risistema(sottoalbero di radice x)
```

Implementazione in loco

Da un array, è possibile creare un heap senza l'uso di memoria aggiuntiva, seguendo lo schema:



```
PROCEDURA risistema(Array A[0...n-1], intero r, intero l)
  v <- r           // v: posizione in esame
  x <- A[v]        // x: chiave da sistemare (per semplicità solo chiave)
  daCollocare <- true
  DO
    IF 2 * v + 1 >= l THEN
      daCollocare <- false
    ELSE
```

```

        u <- indice del figlio di v di valore max **
    IF A[u] > x THEN
        A[v] <- A[u]
        v <- u
    ELSE
        daCollocare <- false
    WHILE daCollocare
        A[v] <- x
```

```

    **:
    u <- 2*u + 1
    IF u+1 < l AND A[u+1] > A[u] THEN
        u <- u + 1
```

Nuova procedura **creaHeap**

```

PROCEDURA creaHeap(Array A[0...n-1])
    FOR i <- n/2 DOWNTO 0 DO
        risistema(A, i, n)
```

Algoritmo **heapSort** finale

```

ALGORITMO creaHeap(Array A[0...n-1])
    creaHeap(A)
    FOR l <- n -1 DOWNTO 1 DO
        scambia A[0] con A[l]
        risistema(A, 0, l)
```

Riassumendo

- Algoritmo di ordinamento in loco
- nessuna memoria aggiuntiva
- $\#CFR = \Theta(n \log n)$
- Tempo $\Theta(n \log n)$ se i confronti costano $O(1)$
- non stabile

Algoritmi di ordinamento basati su confronti

Problema dell’ordinamento

input: n elementi $x_1, x_2, \dots, x_n$ provenienti da un dominio D su cui è definita una relazione $\leq$ di ordine totale

output: sequenza $x_{j_1}, x_{j_2}, \dots, x_{j_n}$ con $(j_1, j_2, \dots, j_n)$ permutazione di $(1, 2, \dots, n)$ tale che $x_{j_1} \leq x_{j_2} \leq \dots \leq x_{j_n}$

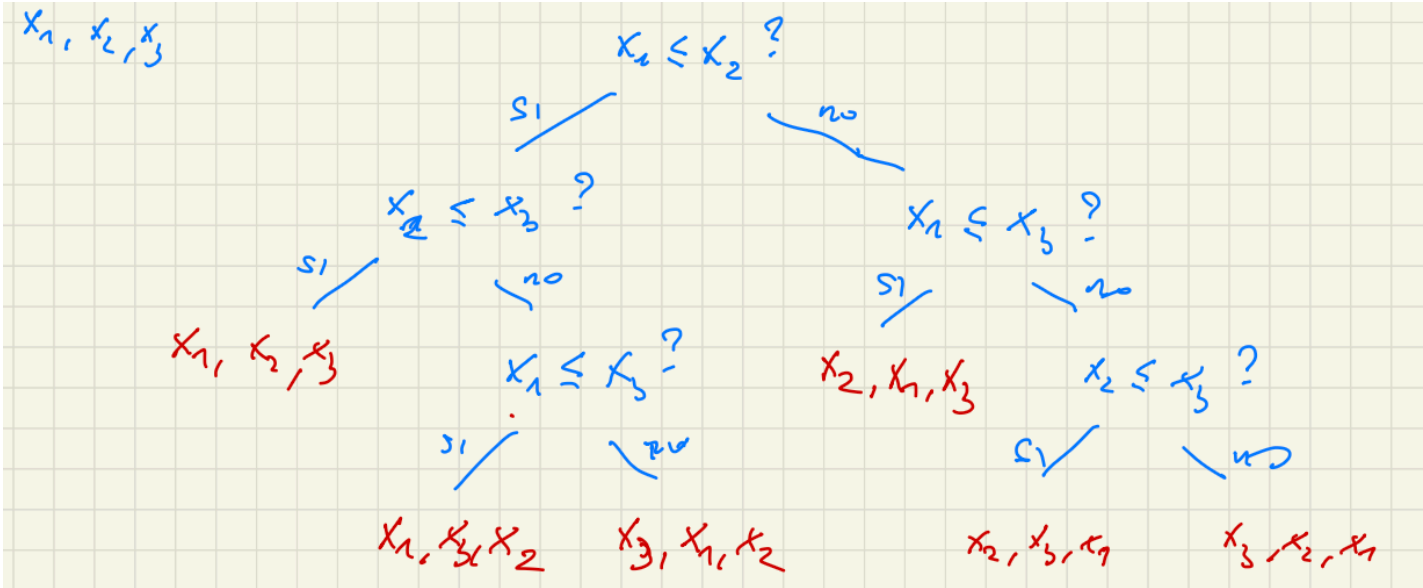
per n elementi, $n!$ permutazioni

Per i vari algoritmi visti fino d’ora

Algoritmo	Numero confronti	Spazio	Note	Stabile
selectionSort	$\Theta(n^2)$ sempre	$O(1)$	in loco	No
insertionSort	$\Theta(n^2)$ caso peggiore $n - 1$ array già ordinato	$O(1)$	in loco	Sì

Algoritmo	Numero confronti	Spazio	Note	Stabile
<code>bubbleSort</code>	$\Theta(n^2)$ caso peggiore $n - 1$ array già ordinato	$O(1)$	in loco	Sì
<code>mergeSort</code>	$\Theta(n \log n)$	$O(n)$	vettore ausiliario $\Theta(n)$ stack ricorsione $\Theta(\log n)$	Sì
<code>quickSort</code>	$\Theta(n^2)$ nel caso peggiore $\Theta(n \log n)$ caso migliore $\approx 1.39n \log_2 n$ in media	$\Theta(n)$ $\Theta(\log n)$	in loco spazio $O(1)$ più stack ricorsione: - $\Theta(n)$ algoritmo base - $\Theta(\log n)$ algoritmo migliorato	No
<code>heapSort</code>	$\Theta(n \log n)$	$O(1)$	in loco	No

è possibile ordinare array n elementi utilizzando un numero di confronti tra chiavi che cresca meno di $n \log n$?



$$\text{altezza} \geq \log_2(\text{\#nodi}) \geq \log_2(\text{\#foglie}) \geq \log_2(n!)$$

→ numero di confronti $\geq \log_2(n!)$

è noto che $n! \approx \sqrt{2\pi n} \cdot \left(\frac{n}{e}\right)^n$ detta *approssimazione di Stirling*

Dunque:

$$\begin{aligned} \log_2(n!) &\approx \log_2 \left(\sqrt{2\pi n} \cdot \left(\frac{n}{e}\right)^n \right) \\ &= \log_2 \sqrt{2\pi} + \log_2 \sqrt{n} + \log_2 \left(\frac{n}{e}\right)^n \\ &= \log_2 \sqrt{2\pi} + \frac{1}{2} \log_2 n + n \log_2 n - n \log_2 e \\ &= \Theta(n \log n) \end{aligned}$$